

SESSION - 11

Client-Side Library



Table of Contents

Client-Side Libraries *2*
Structure of Client-Side Libraries *2*
Organizing Client-Side Libraries *3*
Referencing Client-Side Libraries *3*
Exercise 1: Add Content to a page **4**
Exercise 2: Create a client-side library **5**
Exercise 3: Add a client-side library to a page component **7**
Exercise 4: Add a client-side library to a template **10**

Client-Side Libraries

The standard way to include a client-side library (a JS or a CSS file) in the HTML of a page is to include a <script> or <link> tag in the Java Server Page (JSP) containing the path to the JS/CSS file. Although this approach works in AEM, it can lead to problems when pages and their constituent components become complex. In such cases, multiple copies of the same JS library may be included in the final HTML output.
To avoid this issue, AEM uses client-side library folders to organize the client-side libraries logically. The basic goals of client-side libraries are to:

- Store CSS/JS in small discrete files for easier development and maintenance
- Manage dependencies on third-party frameworks in an organized fashion
- Minimize the number of client-side requests by concatenating CSS/JS into one or two requests
- Minimize CSS/JS that is delivered to optimize speed/performance of a site

Structure of Client-Side Libraries

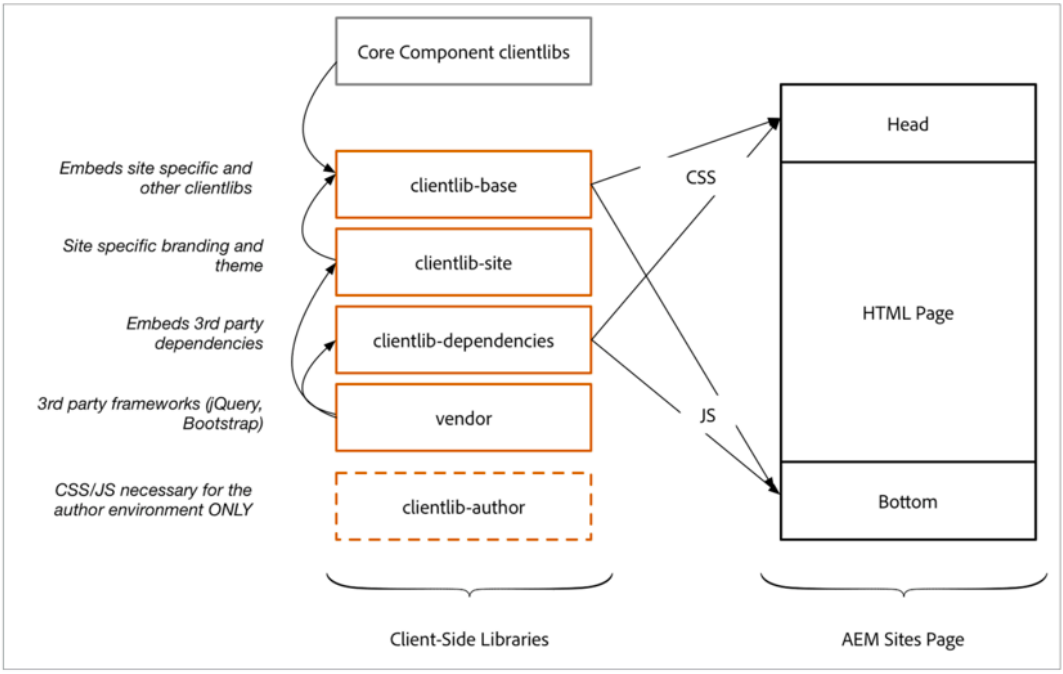
A client-side library folder is a repository node of type cq:ClientLibraryFolder. The typical node structure of a client library in AEM is:

```
[cq:ClientLibraryFolder] - jcr:primaryType
- categories (String[])
- embed (String[])
- dependencies (String[])
+ css.txt (nt:file)
+ js.txt (nt:file)
```

Each cq:ClientLibraryFolder is populated with a set of JS and/or CSS files, along with a few supporting files. You can configure cq:ClientLibraryFolder by using the following properties:

- **categories:** This property helps identify the categories into which the set of JS and/or CSS files within the cq:ClientLibraryFolder belong. The categories property is multi-valued and enables a library folder to be a part of more than one category.
- **dependencies:** This property provides a list of other client library categories on which the library folder depends. For example, given two cq:ClientLibraryFolder nodes F and G, if a file in F requires another file in G to function properly, then at least one of the categories of G should be among the dependencies of F.
- **embed:** This property is used to embed code from other libraries. If node F embeds nodes G and H, the resulting HTML will be a concatenation of content from nodes G and H.
- **allowProxy:** If a client library is located under /apps, the allowProxy property allows access to it through the proxy servlet.
- **js.txt/css.txt: This file helps identify the source files to merge in the generated JS and/or CSS files.**

The below diagram explains the client-side libraries' convention followed in the We.Retail site (that is available out-of-the-box in AEM), and this convention can be applied to most sites' implementations:



Organizing Client-Side Libraries

By default, the cq:ClientLibraryFolder nodes can exist anywhere within the /apps and /libs subtrees of the repository. The common locations for clientlibs are:
Site-level (/apps/<your-project>/clientlibs)

Component-level (/apps/<your-project>/components)

Site-Level
The clientlibs at the site-level:

- Are used for CSS/JS for site specific design elements
- Have CSS at the top of page and JS at the bottom of page
    - Examples include responsive design, dependencies, embedding structure components, and vendor code

Component-Level
The clientlibs at the component-level:

- Are component-specific CSS/JS
- Can be embedded into the site design

Referencing Client-Side Libraries

You can include the client-side libraries in HTML Template Language (HTL), which is the preferred technology for developing AEM sites. However, you can also use JSP.
In HTL, client-side libraries are loaded through a helper template provided by AEM. You can access the templates through `data-sly-use`. The three templates, css, js, and all can be called through `data-sly-call`:

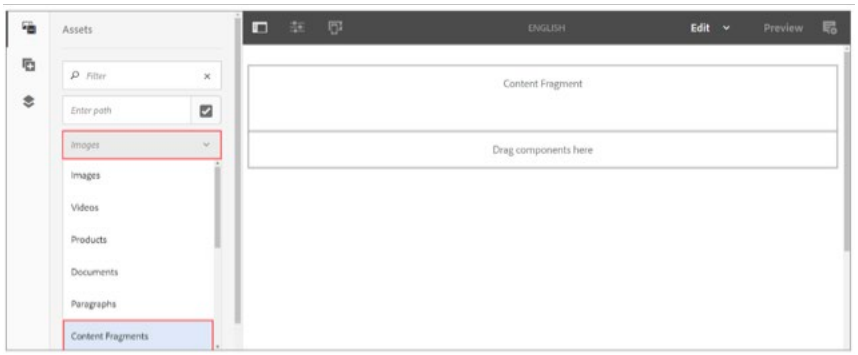
- css: Loads only the CSS files of the referenced client libraries.
- js: Loads only the JavaScript files of the referenced client libraries.
- all: Loads all files of the referenced client libraries (both CSS and JavaScript).

Each helper template expects a categories option for referencing the desired client libraries. This option can be either an array of string values or a string containing a comma-separated values (CSV) list.

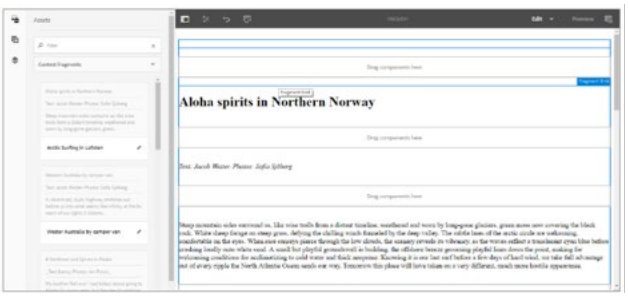
Exercise 1: Add Content to a page

In this exercise, you will add some content to a page. After completing this exercise, you will see the content without a design. In later exercises, you will apply the We.Train design and you will see the content transform to the We.Train design standards.

1. In your AEM author instance, navigate to Sites > We.Train > English page.
2. Click the English page thumbnail, and click Edit (e) on the header bar to open the English page.
3. In the left panel, click the Components icon. The available components are displayed.
4. Drag and drop the Content Fragment component onto the layout container.
5. In the left panel, click the Assets icon. The available assets are displayed.
6. Filter the assets based on Content Fragments by clicking the drop-down menu and selecting Content Fragments, as shown:



7. Drag and drop the Arctic Surfing in Lofoten fragment onto the Content Fragment component. Notice how the content has no design currently.



Exercise 2: Create a client-side library

In this exercise, you will create a base client-side library. This base library will help compile the site's css/js into a single client library.

1. Navigate to Adobe Experience Manager > Tools > CRXDE Lite. The CRXDE Lite page opens.
2. Navigate to the /apps/training folder.
3. Right-click the training folder and click Create > Create Folder. The Create Folder dialog box opens.
4. Type clientlibs in the Name field and click OK. The folder is created under the training folder.
5. Click Save All.
6. Right-click the clientlibs folder and click Create > Create Node. The Create Node dialog box opens.
7. Update the following fields:

   a. Name: clientlib-base

   b. Type: cq:ClientLibraryFolder
8. Click OK. The folder is created.
9. Click Save All from the actions bar.
10. Select the **clientlib-base** folder, and add the following properties:

Name	Type	Value
categories	String[]	we.train.base
embed	String[]	we.train.site we.train.font-awesome
allowProxy	Boolean	true

Note: To add the below properties, you must add the property, click Add, and then click Save All. Follow the same procedure for other properties.

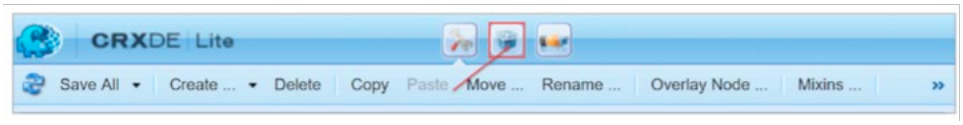
Note: To enter a String Array (String[]), select String from Type field, and then click the Multi button. Remember, the Multi button will be selected until you deselect it. If a dialog box appears with the string array information, click OK.

11. When updating the embed property, the Edit embed dialog should look, as shown:
12. Click Save All from the actions bar.
13. Right-click the **clientlib-base** node and click **Create > Create File**. The **Create File** dialog box opens.
14. Enter **css.txt** in the Name field and click OK. The file is created under the clientlibs folder.
15. Click **Save All** from the actions bar.
16. Perform steps 13 through 15 to create the **js.txt** file.

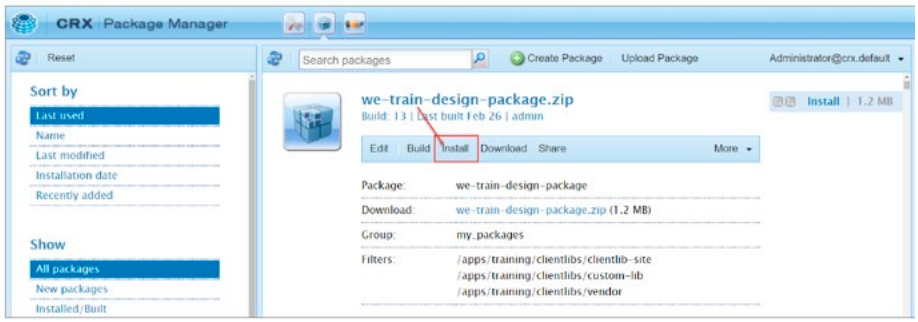
Next, you will upload the **we-train-design-package** to the JCR. This package contains the css/js that you will use in the We.Train site structure.

To install the we-train-design-package:

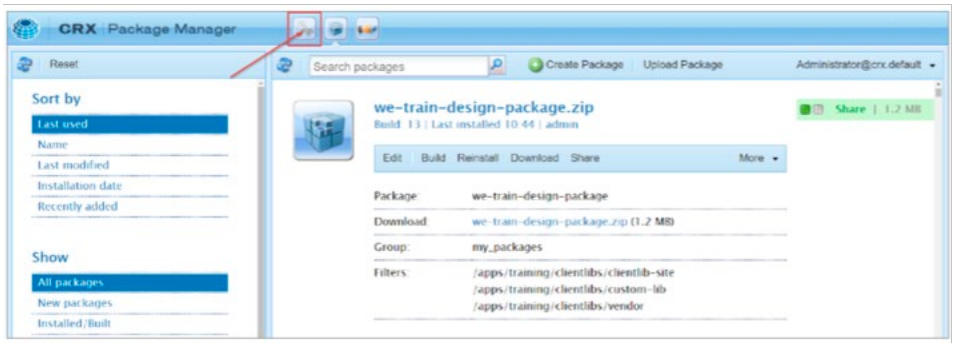
17. From CRXDE Lite, click the Package icon from the header bar, as shown. The Package Manager page opens.



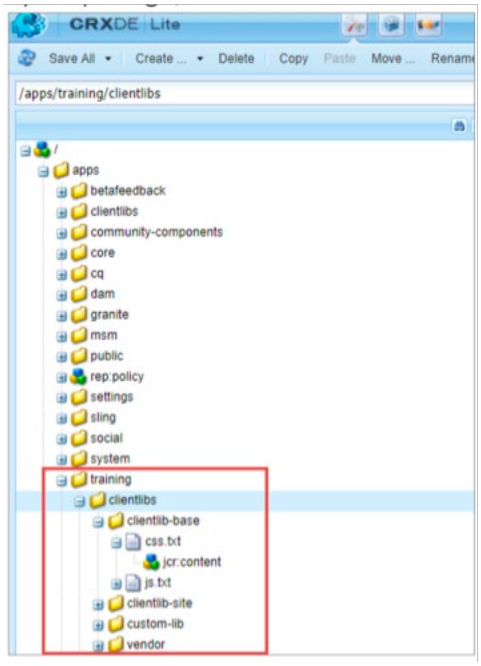
18. Click Upload Package from the actions bar. The Upload Package dialog box opens.
19. Click Browse.
20. Navigate to the Creating Client-side Libraries in Adobe Experience Manager exercise folder provided to you and double-click the **we-train-design-package.zip** package.
21. Click OK in the Upload Package dialog box. The package is uploaded.
22. In the **we-train-design-package.zip** section, click Install, as shown. The Install Package dialog box opens. Notice the filters on the package. This dialog box shows the files and folders that will be installed in your **/apps/training/clientlibs** folder on JCR.



23. Ignore the Advanced Settings and click Install. The package is installed on your instance.
24. Click the Develop icon from the header bar, as shown. The CRXDE Lite page is refreshed.



25. Navigate to the /apps/training/clientlibs folder and observe the client libraries that are installed by the package.



Exercise 3: Add a client-side library to a page component

In this exercise, you will include the client libraries directly in the page component with convenience scripts called customheaderlibs.html and customfooterlibs.html. These scripts enable a developer to hard code a mandatory design to the page component.

1. Click http://localhost:4502/crx/de/index.jsp if you are not already in CRXDE Lite. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/clientlibs/clientlib-base** node. Observe the property **categories=we.train.base.** You will use this category name when adding the clientlib to a page component.

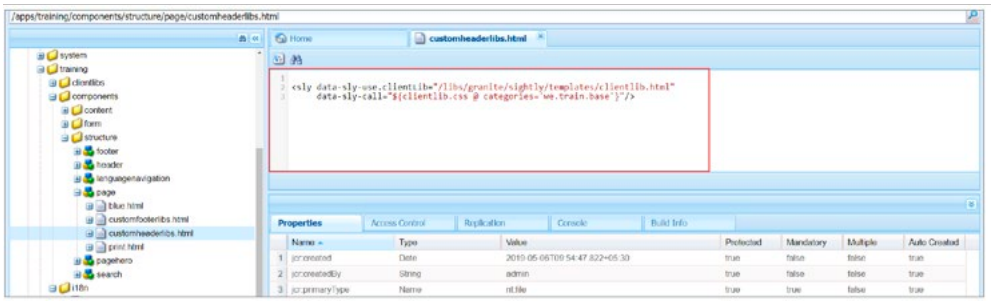
To add the HTML files:

3. In CRXDE Lite, navigate to the /apps/core/wcm/components/page/v2/page component.

4. Right-click the customheaderlibs.html file and click Copy.
5. Navigate to the /apps/training/components/structure/page.
6. Right-click the page node and click Paste to paste the file you copied in step 4.
7. Similarly, copy the customfooterlibs.html file from /apps/core/wcm/components/page/v2/ page folder and paste it to the /apps/training/components/structure/page node.
8. Click Save All.

To add code to the .html files:

9. Navigate to the exercise files, open customheaderlibs.html in any text-pad or notepad, copy the script, and paste it into customheaderlibs.html in CRXDE Lite, as shown:

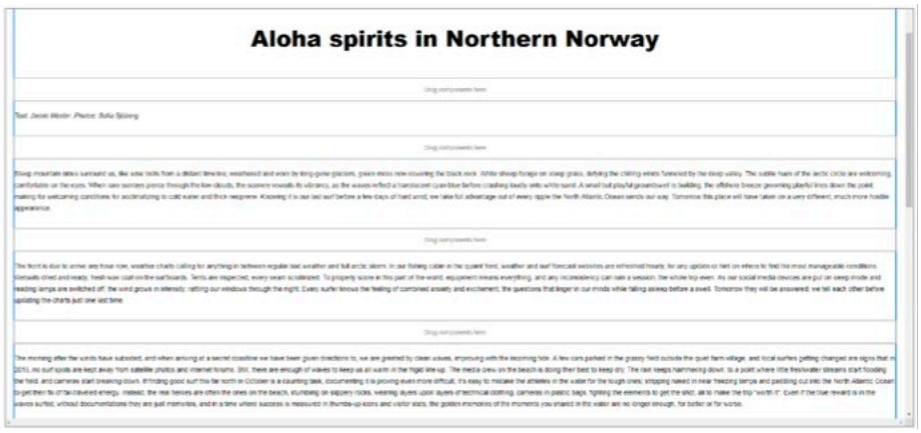


10. Click save all.
11. Repeat step 9 but copy-paste the script from customfooterlibs.html to CRXDE Lite.
12. Click Save All.

You can verify if the CSS and JavaScript client libraries were added to the top and bottom of the page from the HTML source of a page.

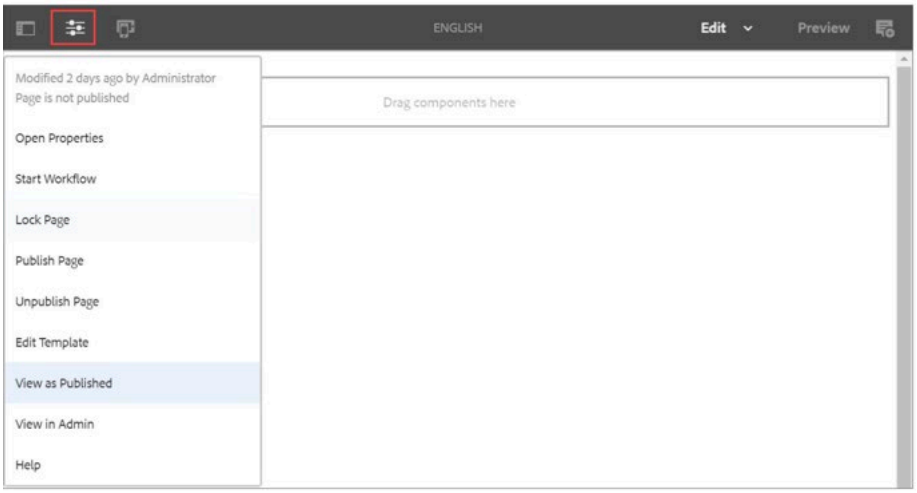
13. Click CRXDE Lite from the header bar. You will be taken to the AEM Navigation page.
14. Click Sites > We.Train site.
15. Click the thumbnail on the English page, and then click Edit (e) from the actions bar. The English page opens in a new tab.

On the English page, you should see that a different design is applied to the content fragment you added earlier.

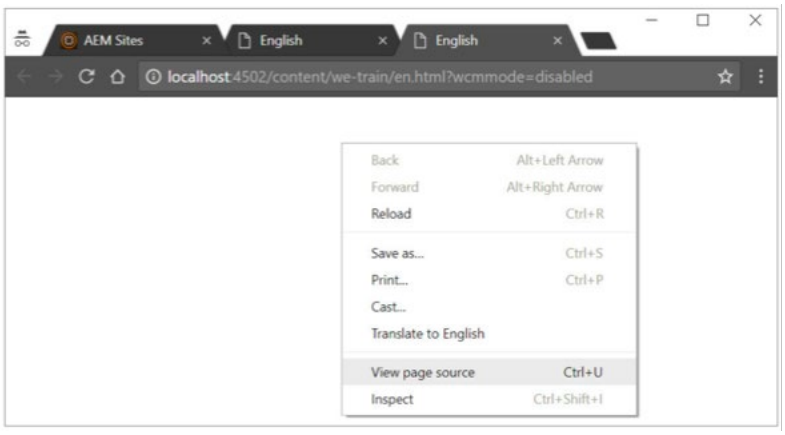


To verify that the client libraries are being loaded:

16. Click the Page Information icon from the page toolbar, and select View as Published from the drop-down menu, as shown. The page opens in a new tab of the browser.



17. Right-click in the English page, and click **View page source**, as shown. The source code opens in a new tab of the browser.



18. Look for the line, `<link rel="stylesheet" href="/etc.clientlibs/training/clientlibs/client-base. css" type="text/css">` in the source code. Notice how the **href** tag points to the clientlibs path in the JCR that confirms the customheaderlibs.html is referenced in the page, as shown:



19. Look for the line, `<script type="text/javascript" src="/etc.clientlibs/training/clientlibs/client-base.js"></script>` in the source code. Notice how the **href** tag points to the clientlibs path in the JCR that confirms the customfooterlibs.html referenced in the page, as shown:



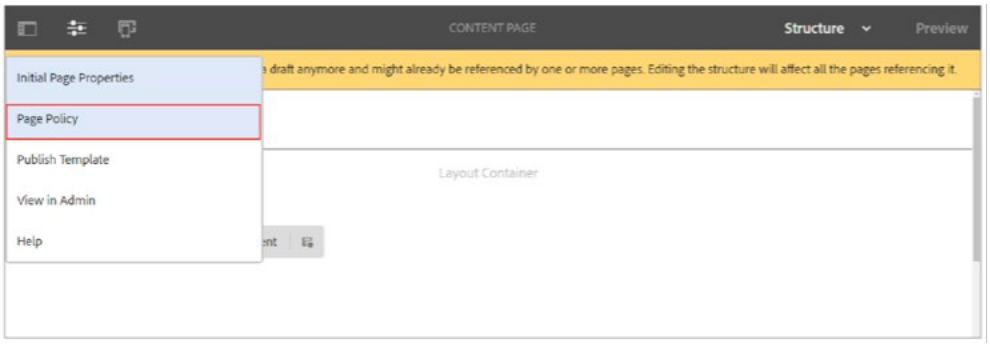
Exercise 4: Add a client-side library to a template

In the previous exercise, you learned how a developer can code in the client libraries directly in the header and footer of a page. If the template editor is adding client libraries to the page, they can do this through the Template Editor. This approach can be useful when page templates use the exact same components and layout, but templates need to have a different design for microsites and multiple tenants with a single development team.

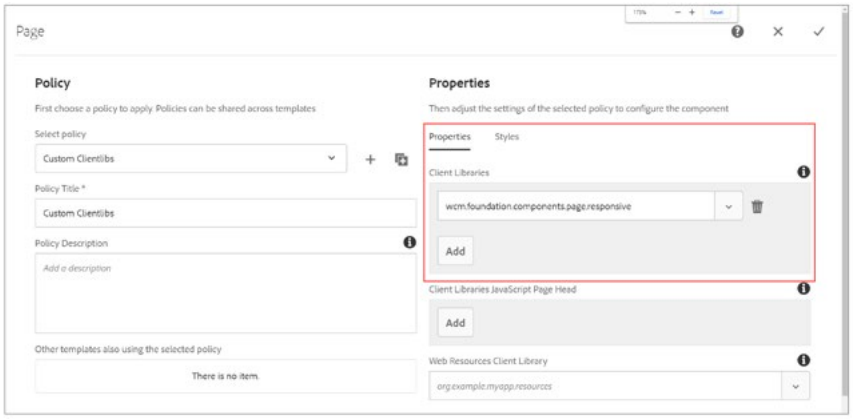
To add a client library to a template:

1. Navigate to the Sites > We.Train > English page in the AEM author instance.
2. Open the English page by clicking Edit (e) in the actions bar, click the Page Information icon from the page toolbar, and select Edit Template from the drop-down menu. The template opens on a new tab.

3. Ensure you are in the content page template, click the Page Information icon from the toolbar, and select Page Policy from the drop-down menu, as shown. The Page Content Policy opens.



4. In the Policy section, enter Custom Clientlibs in the Policy Title field.
5. In the Properties section, notice that the clientlib, wcm.foundation.components.page. responsive, is already added to the template, as shown. It is the default AEM responsive grid. Notice that you can also add new client libraries to the page template.



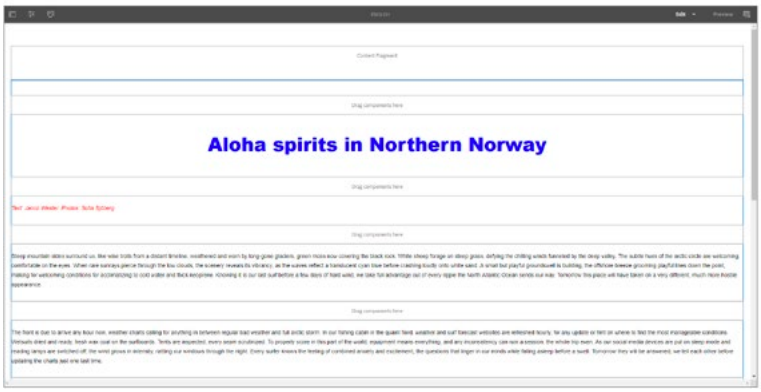
To add the custom clientlib to the template:

6. Click Add and enter we.train.custom.lib in the field.
7. Click Done (checkmark icon in the upper right). The custom clientlib is added to the template.

Note: The Content Page template is already enabled, which means the changes you make affect all pages that were built from the template.

Caution: The design changes you made are only on the author instance. If you want this change to be replicated to the publish instance, you must republish this template.

8. Navigate to the **We.Train** > **English** page, and refresh the browser. In the content fragment you added earlier, you should now see *h1* tags in blue and *p* tags in red. This is the result of the client library being added to the template.



9. You can verify that the client library was added by checking the source of the page. Click the Page Information icon from the page toolbar and click View as Published. The page opens in a new browser tab.
10. Right-click in the English page and click View page source. The source code opens in a new browser tab.
11. Look for the line, <link rel="stylesheet" href="/etc.clientlibs/training/clientlibs/custom-lib. css" type="text/css"> in the source code. Notice how the href tag points to the clientlibs path in JCR that confirms the custom clientlibs is added to the page, as shown. Click the links to examine its content.

